Université de Lorraine - Master 2 Sciences des Données (M2SDD)

Projet commun : Algorithmique · Programmation · Mathématiques pour l'informatique

Année universitaire 2025-2026

Projet Réussite

Dossier Technique

Prédiction de la réussite d'un apprenant
à partir de ses traces numériques sur la plateforme ARCHE

Auteur : Matthieu PELINGRE
Email : matthieu.pelingre1@etu.univ-lorraine.fr
Dépôt : github.com/M2SDD/projet-reussite
Date : Avril 2026

Logiciel développé en Python 3 · Tkinter · scikit-learn · pandas · matplotlib

Table des matières

1 Architecture générale du logiciel **2**
1.1 Vue d'ensemble . . . 2
1.1.1 Structure des répertoires . . . 2
1.2 Architecture Modèle-Vue-Contrôleur (MVC) . . . 3
1.3 Patrons de conception utilisés . . . 3

2 Diagramme de classes **5**
2.1 Hiérarchie des modèles de prédiction . . . 5
2.2 Pipeline de préparation des données . . . 6
2.3 Module d'évaluation . . . 6

3 Conception de l'interface utilisateur **7**
3.1 Principes de conception . . . 7
3.2 Onglet 1 : Données . . . 7
3.3 Onglet 2 : Prétraitement . . . 8
3.4 Onglet 3 : Modélisation . . . 9
3.5 Onglet 4 : Évaluation . . . 9
3.6 Gestion asynchrone . . . 10

4 Principaux algorithmes **11**
4.1 Extraction de variables (Feature Engineering) . . . 11
4.2 Sélection de variables . . . 12
4.2.1 Phase 1 : Pré-filtrage . . . 12
4.2.2 Phase 2 : Sélection avancée (Ensemble Feature Selection) . . . 12
4.3 Entraînement des modèles : Méthode Patron . . . 12
4.4 Métriques d'évaluation . . . 13

5 Utilisation de l'intelligence artificielle générative **14**
5.1 Rôle de l'IA dans le développement . . . 14
5.2 Limites et responsabilité . . . 14

Conclusion **15**

Chapitre 1

Architecture générale du logiciel

1.1 Vue d'ensemble

Le logiciel **Projet Réussite** est une application orientée objet de bureau développée en Python 3. L'application réalise un pipeline complet allant du chargement des données brutes jusqu'à la comparaison de modèles de prédiction, avec une interface graphique `Tkinter`.

L'ensemble du code source est organisé dans le répertoire `src/`, structuré en sous-paquets thématiques. Le point d'entrée unique est `main.py`, à la racine du projet.

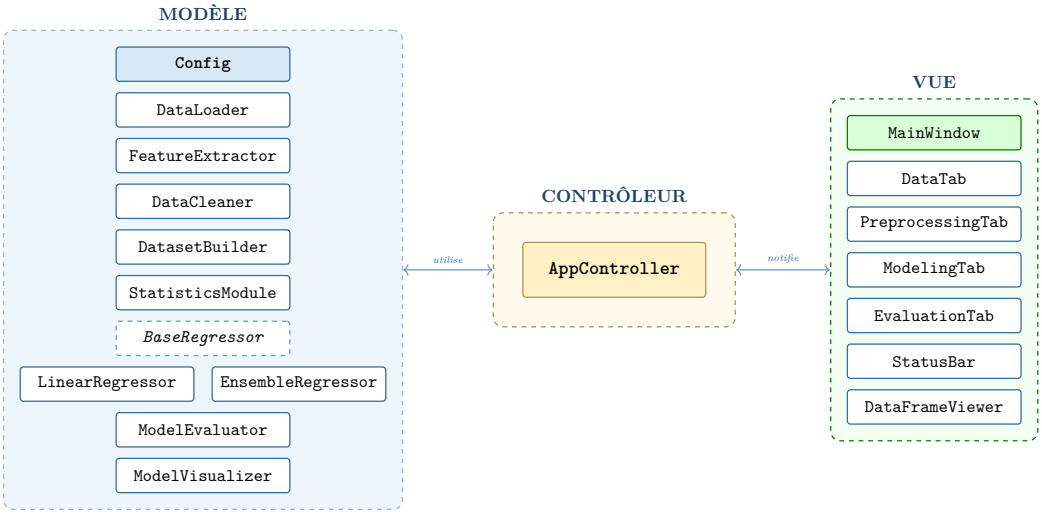
1.1.1 Structure des répertoires

```
projet_reussite/
|-- main.py                        # Point d'entree
|-- data/
|   |-- logs_info_25_pseudo.csv
|   `-- notes_info_25_pseudo.csv
`-- src/
    |-- config.py                  # Configuration centralisee
    |-- data/
    |   |-- data_loader.py         # Chargement et validation
    |   |                          # CSV
    |   |-- feature_extractor.py   # Feature Engineering
    |   |-- data_cleaner.py        # Nettoyage et selection de
    |   |                          # variables
    |   |-- dataset_builder.py     # Orchestrateur ETL
    |   `-- statistics_module.py   # Statistiques descriptives
    |-- models/
    |   |-- base_regressor.py      # Classe abstraite de base
    |   |-- linear_regressor.py
    |   `-- ensemble_regressor.py
    |-- evaluation/
    |   `-- model_evaluator.py
    |-- visualization/
    |   `-- model_visualizer.py
    `-- interface/
        |-- app_controller.py      # Controleur MVC
        |-- main_window.py         # Fenetre principale
        |-- tabs/                  # Quatre onglets (Vues MVC)
        `-- widgets/               # Composants reutilisables
```

Listing 1.1 – Arborescence du projet

1.2 Architecture Modèle-Vue-Contrôleur (MVC)

L'application suit une architecture **Modèle-Vue-Contrôleur** (MVC). Ce choix architectural répond à la contrainte d'une interface graphique `Tkinter` tout en séparant clairement la logique métier de la présentation.



Modèle Regroupe toutes les classes métier : chargement des données (`DataLoader`), extraction de variables (`FeatureExtractor`), nettoyage (`DataCleaner`), orchestration (`DatasetBuilder`), modèles prédictifs (`BaseRegressor` et ses sous-classes) et évaluation (`ModelEvaluator`). Ces classes sont indépendantes de l'interface graphique.

Contrôleur La classe `AppController` constitue l'unique point de communication entre la vue et le modèle. Elle maintient l'état global de l'application (données chargées, dataset construit, modèles entraînés) et expose des méthodes métier de haut niveau. Les onglets ne font *jamais* d'appels directs aux classes du modèle.

Vue `MainWindow` crée le `ttk.Notebook` et les quatre onglets. Chaque onglet est une classe autonome gérant ses propres widgets Tkinter. La vue ne contient aucune logique métier.

1.3 Patrons de conception utilisés

Trois patrons de conception (*design patterns*) structurent l'application.

Patron 1 : `BaseRegressor` (Template Method)

La classe abstraite `BaseRegressor` définit le squelette de l'algorithme d'entraînement dans sa méthode publique `fit()` : validation des données, mise à l'échelle, puis appel à `_fit()`. La méthode `_fit()` est abstraite et doit être implémentée par chaque sous-classe (`LinearRegressor`, `EnsembleRegressor`). Cela garantit que toutes les validations et le scaling sont toujours effectués, quelle que soit l'implémentation concrète.

Patron 2 : `DatasetBuilder` (Facade Pattern)

`DatasetBuilder` orchestre trois composants (`DataLoader`, `FeatureExtractor`, `DataCleaner`) en exposant une interface unifiée via la méthode `build_dataset()`. Le contrôleur n'a pas besoin de connaître les détails de chaque étape du pipeline.

Patron 3 : `DataCleaner.select_features()` (Strategy Pattern)

La méthode `select_features()` de `DataCleaner` accepte une liste de méthodes de sélection (`'linear'`, `'mutual_info'`, `'importance'`, `'rfe'`) et délègue à la méthode correspondante. Cette approche permet de combiner plusieurs stratégies de sélection sans modifier le code appelant.

Chapitre 2

Diagramme de classes

2.1 Hiérarchie des modèles de prédiction

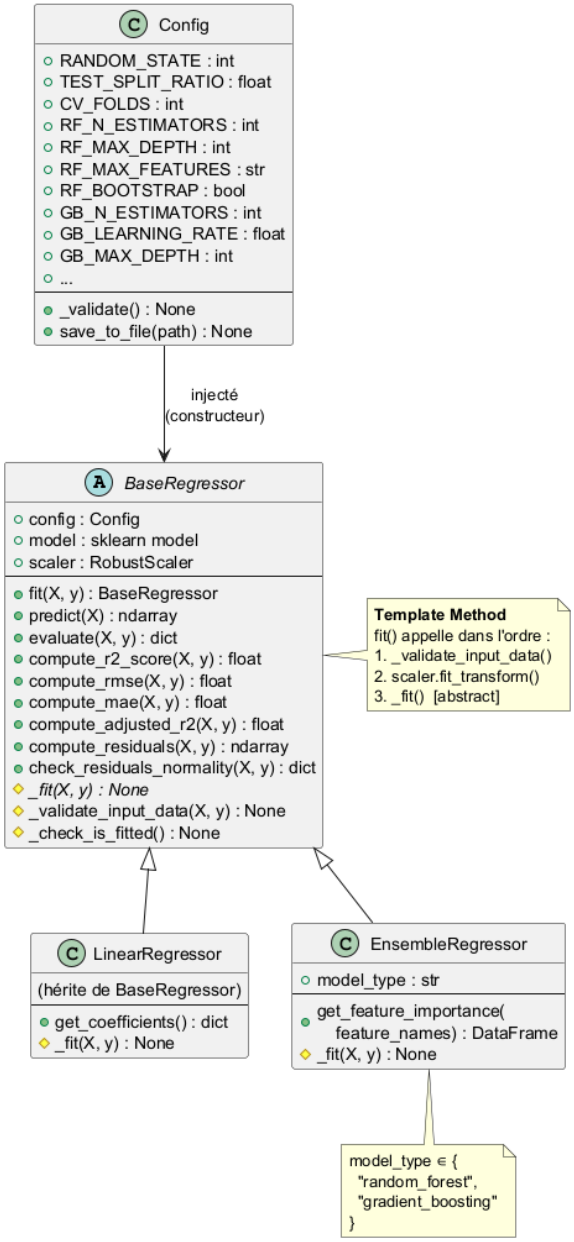
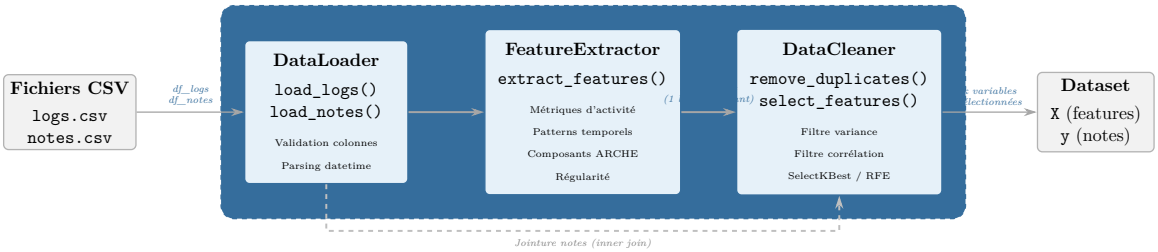


Figure 2.1 – Hiérarchie d'héritage des classes de modèles de prédiction

La relation d'héritage entre `BaseRegressor` et ses sous-classes illustre le patron *Template Method* : la logique commune (scaling, validation, calcul des métriques) est

centralisée dans la classe abstraite, tandis que chaque sous-classe n'implémente que la logique qui lui est propre via la méthode `_fit()`.

2.2 Pipeline de préparation des données



Le `DatasetBuilder` encapsule les trois composants du pipeline de données. Le contrôleur (`AppController`) invoque uniquement `build_dataset()` et reçoit directement les matrices `X` et `y` prêtes à l'emploi.

2.3 Module d'évaluation

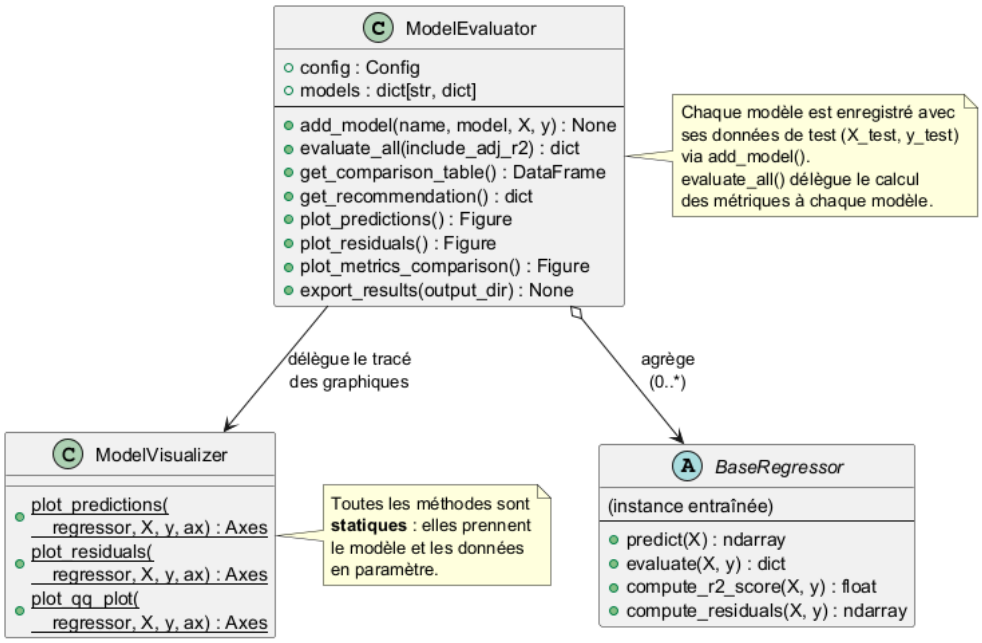


Figure 2.2 – Classes du module d'évaluation et leurs relations

`ModelEvaluator` centralise la comparaison des modèles enregistrés et délègue la génération des graphiques à `ModelVisualizer`, dont toutes les méthodes sont statiques.

Chapitre 3

Conception de l'interface utilisateur

3.1 Principes de conception

L'interface graphique est construite avec **Tkinter/ttk** (bibliothèque standard Python). Elle est organisée autour d'un `ttk.Notebook` à quatre onglets, activés progressivement au fil du pipeline. Ce mécanisme d'activation progressive guide l'utilisateur et prévient toute erreur d'utilisation.

N°	Onglet	Condition d'activation	Classe
0	Données	Toujours actif	`DataTab`
1	Prétraitement	Après chargement des CSV	`PreprocessingTab`
2	Modélisation	Après construction du dataset	`ModelingTab`
3	Évaluation	Après entraînement d'un modèle	`EvaluationTab`

3.2 Onglet 1 : Données

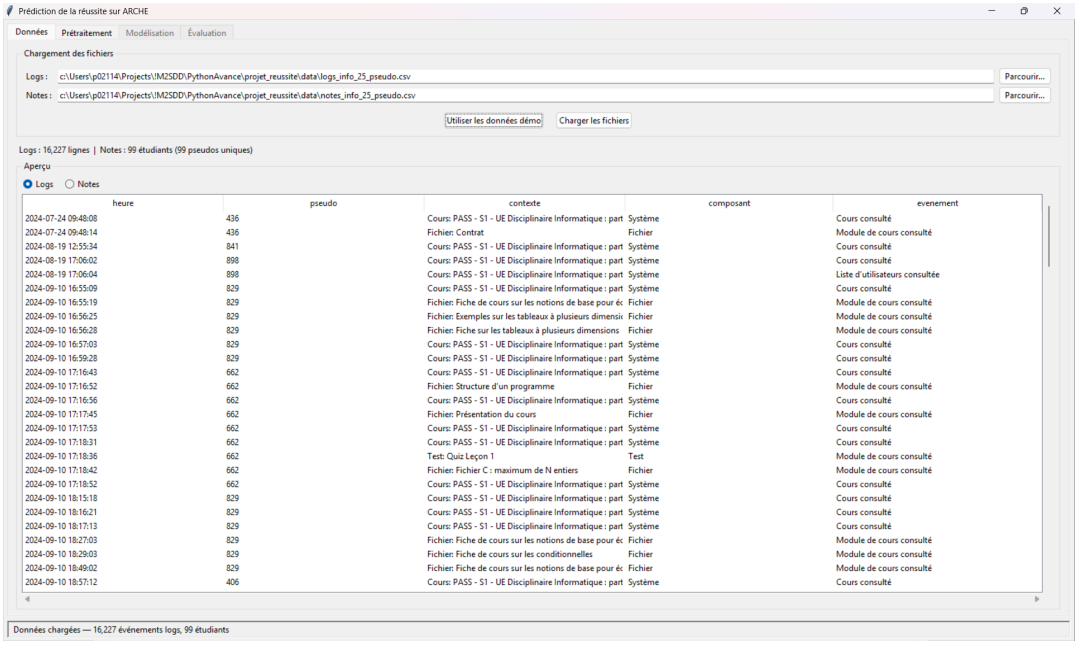


Figure 3.1 – Onglet *Données* : Chargement et aperçu des fichiers CSV

L'onglet **Données** permet à l'utilisateur de :

— sélectionner les deux fichiers CSV (logs et notes) via un dialogue système, ou charger directement les fichiers de démonstration inclus.

— visualiser un aperçu tabulaire (`DataFrameViewer`) de chaque fichier, basculé par boutons radio (Logs / Notes).

— consulter un résumé (nombre de lignes, d'étudiants, de pseudos uniques).

Le chargement s'effectue dans un **thread secondaire** (module `threading`) pour éviter de bloquer l'interface lors du traitement de fichiers volumineux.

3.3 Onglet 2 : Prétraitement

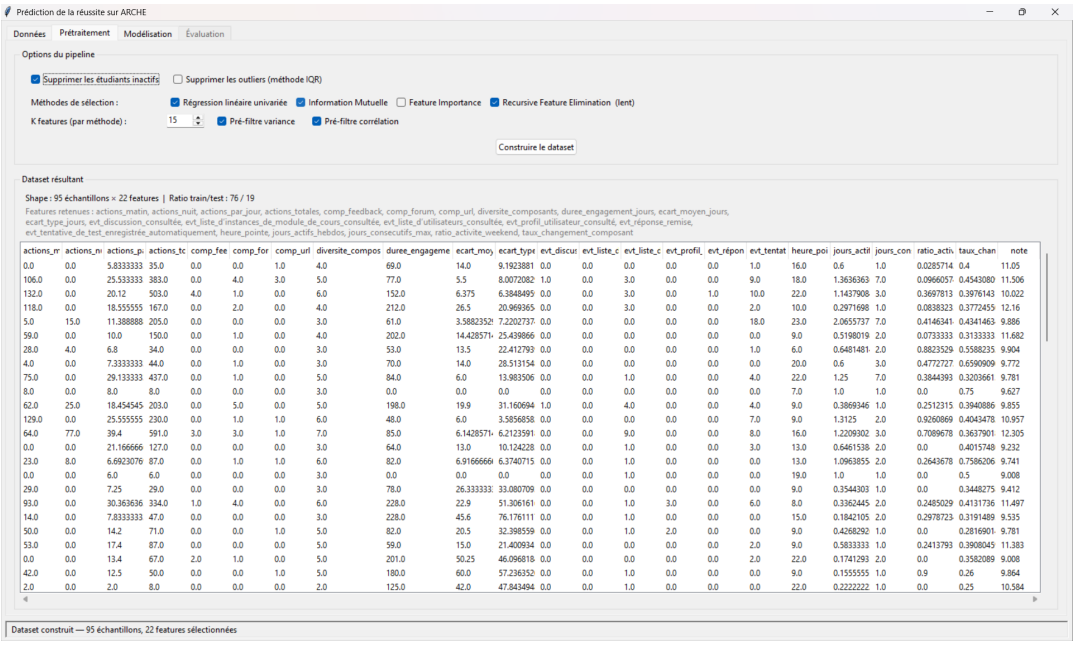


Figure 3.2 – Onglet *Prétraitement* : Configuration du pipeline et dataset résultant

L'onglet **Prétraitement** expose toutes les options configurables du pipeline `DatasetBuilder` :

— **Suppression des étudiants inactifs :** si coché, les étudiants présents dans les notes mais absents des logs sont exclus via un `inner join` entre les logs et les notes. Sinon, les variables manquantes sont remplacées par 0.

— **Suppression des outliers (IQR) :** suppression des observations statistiquement aberrantes (si la majorité des variables sont aberrantes).

— **Méthodes de sélection de variables :** quatre méthodes combinables (Test F, information mutuelle, importance Random Forest, RFE).

— **Paramètre k :** nombre de variables à retenir par méthode.

— **Pré-filtres :** filtre de variance quasi-nulle et filtre de colinéarité.

3.4 Onglet 3 : Modélisation

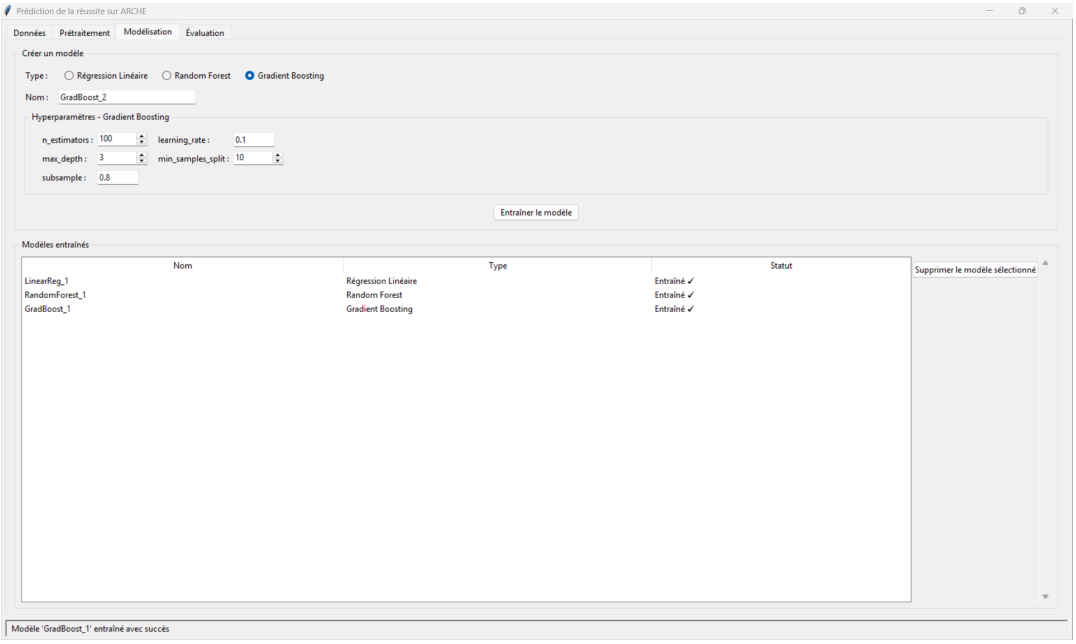


Figure 3.3 – Onglet *Modélisation* – Création et entraînement des modèles

L'onglet **Modélisation** permet de choisir le type de modèle, de le nommer, de configurer ses hyperparamètres via des spinbox dédiées, puis de lancer l'entraînement. Plusieurs modèles peuvent être entraînés simultanément pour comparaison.

3.5 Onglet 4 : Évaluation

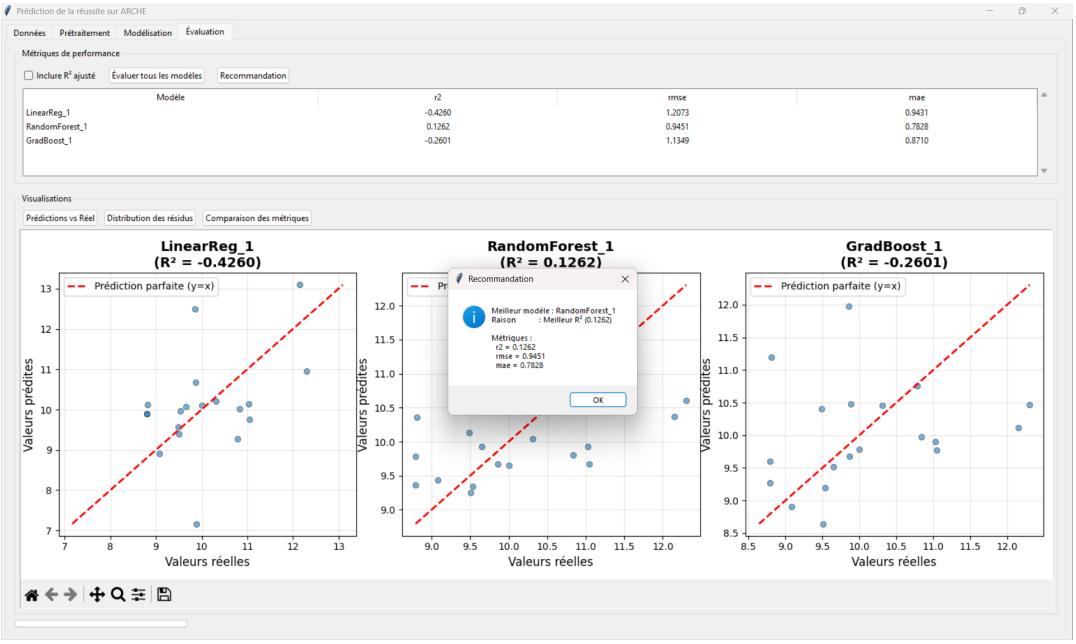


Figure 3.4 – Onglet *Évaluation* – Comparaison des modèles et visualisations

L'onglet **Évaluation** offre :

— un tableau de comparaison des métriques (R^2, RMSE, MAE).

— trois sous-onglets de visualisation : prédictions vs réel, distribution des résidus, comparaison des métriques par modèle.

— un bouton *Recommandation* analysant automatiquement les métriques.

— l'export de tous les résultats (CSV et images PNG) vers `output/`.

3.6 Gestion asynchrone

Toutes les opérations longues sont exécutées dans un thread secondaire afin de préserver la réactivité de l'interface Tkinter :

```python
def _run_thread(self, worker, on_success, on_error=None) -> None:
    def target():
        try:
            result = worker()
            # Callback planifie dans le thread principal (Tk)
            self.frame.after(0, on_success, result)
        except Exception as exc:
            if on_error:
                self.frame.after(0, on_error, str(exc))

    threading.Thread(target=target, daemon=True).start()
```

Listing 3.1 – Patron de thread commun aux onglets

L'utilisation de `frame.after(0, callback)` garantit que la mise à jour de l'interface se fait toujours dans le thread principal de Tkinter, conformément aux contraintes de la bibliothèque.

Chapitre 4

Principaux algorithmes

4.1 Extraction de variables (Feature Engineering)

La classe `FeatureExtractor` transforme un fichier de logs brut (une ligne par événement) en un tableau récapitulatif par étudiant. L'extraction est découpée en cinq sous-méthodes appelées séquentiellement par `extract_features()`.

Méthode	Variables produites	Nb.
`compute_activity_metrics`	Actions totales, jours actifs, durée d'engagement, sessions	4
`compute_temporal_patterns`	Heure de pointe, ratio week-end, tranches horaires, ...	8
`compute_component_features`	Comptage par composant ARCHE (`comp_*`)	variable
`compute_event_type_features`	Comptage par type d'événement (`evt_*`)	variable
`compute_consistency_features`	Jours consécutifs max, écarts inter-sessions, fréquence	4
`compute_interaction_depth`	Actions par jour, diversité, taux de changement	4

L'algorithme pour calculer la plus longue série de jours consécutifs d'activité illustre la complexité du feature engineering :

```python
def calculate_streak_days(dates):
    if len(dates) == 0:
        return 0
    if len(dates) == 1:
        return 1

    max_streak = 1
    current_streak = 1

    dates = pd.to_datetime(pd.Series(dates))
    for i in range(1, len(dates)):
        diff = (dates[i] - dates[i-1]).days
        if diff == 1:
            current_streak += 1
            max_streak = max(max_streak, current_streak)
        else:
        current_streak = 1

    return max_streak
```

Listing 4.1 – Calcul du plus grand nombre de jours d'activité successif

4.2 Sélection de variables

La sélection des variables s'effectue en deux phases dans `DataCleaner`.

4.2.1 Phase 1 : Pré-filtrage

1. **Filtre de variance quasi-nulle** (`VarianceThreshold`) : supprime les colonnes dont la variance est inférieure à 0,01, typiquement les composants ARCHE que personne n'utilise.
2. **Filtre de colinéarité :** pour chaque paire de variables dont $|r| > 0{,}85$, la variable à plus faible variance est supprimée.

4.2.2 Phase 2 : Sélection avancée (Ensemble Feature Selection)

Jusqu'à quatre méthodes peuvent être combinées. L'union des ensembles de variables sélectionnées par chaque méthode est retenue.

Méthode	Classe sklearn	Principe
`linear`	`SelectKBest(f_regression)`	Corrélation linéaire (Test F)
`mutual_info`	`SelectKBest(mutual_info_regression)`	Information mutuelle
`importance`	`RandomForestRegressor`	Importance (Gini / MSE)
`rfe`	`RFE(RandomForestRegressor)`	Élimination récursive

4.3 Entraînement des modèles : Méthode Patron

La méthode publique `fit()` de `BaseRegressor` définit le squelette invariant de l'entraînement :

```python
def fit(self, X, y) -> 'BaseRegressor':
    # 1. Validation des entrees (identique pour tous les modeles)
    self._validate_input_data(X, y)
    # 2. Standardisation (RobustScaler, identique pour tous)
    X = self.scaler.fit_transform(X)
    # 3. Entrainement specifique (implementation dans les sous-classes)
    self._fit(X, y)
    return self

@abstractmethod
def _fit(self, X, y) -> None:
    """A implementer dans chaque sous-classe."""
    pass
```

Listing 4.2 – Template Method dans BaseRegressor

Le `RobustScaler` a été choisi car les données de logs présentent de fortes disparités (un étudiant peut avoir 10 actions, un autre plus de mille 1 000). Contrairement au `StandardScaler`, il utilise la médiane et l'écart interquartile, ce qui le rend robuste aux valeurs extrêmes.

4.4 Métriques d'évaluation

Trois métriques principales sont calculées pour chaque modèle :

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad \in (-\infty,\, 1] \tag{4.1}$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2} \tag{4.2}$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i| \tag{4.3}$$

Le R^2 **ajusté** est disponible en option :

$$R^2_{\text{adj}} = 1 - (1 - R^2) \cdot \frac{n-1}{n-p-1} \tag{4.4}$$

où n est le nombre d'observations et p le nombre de variables explicatives.

Chapitre 5

Utilisation de l'intelligence artificielle générative

Dans un souci de transparence académique, ce dossier mentionne explicitement le recours à un outil d'intelligence artificielle générative, indispensable afin de pouvoir rendre ce dossier dans les délais impartis.

5.1 Rôle de l'IA dans le développement

L'assistant **Claude Code** (Anthropic, modèle Sonnet 4.6) a été utilisé tout au long du projet pour :

— l'aide à la rédaction des *docstrings* et de la documentation.

— la review de code et la rédaction des tests unitaires.

— génération de maquettes et plan d'actions pour l'implémentation de l'interface graphique.

— relecture des présents dossiers.

— la génération des diagrammes TikZ et UML.

5.2 Limites et responsabilité

L'ensemble du code source a été **relu, compris et validé** par l'auteur avant intégration. Les choix algorithmiques, architecturaux et d'analyse restent de la responsabilité de l'auteur.

Remarque

L'utilisation de l'IA générative est un sujet d'actualité dans l'enseignement supérieur. Considérant que la transparence est préférable à la dissimulation, et que la maîtrise des outils d'IA fait partie des compétences attendues d'un Master en Sciences des Données, ce recours est ici déclaré explicitement.

Conclusion

Ce dossier technique a présenté la conception du logiciel *Projet Réussite*, développé en Python 3 en respectant les contraintes imposées (POO, sans notebook, Tkinter). L'architecture MVC assure une séparation claire des responsabilités, les patrons de conception (Template Method, Façade, Stratégie) favorisent l'extensibilité du code, et la gestion asynchrone garantit la réactivité de l'interface graphique.

L'analyse des résultats et les limites du projet sont développées dans le **Dossier d'Analyse** associé.